

e-gov-framework

ARC42 Architekturdokumentation

Inhaltsverzeichnis

1	Einführung und Ziele	5
1.1	Aufgabenstellung	5
1.2	Qualitätsziele	5
1.3	Stakeholder	5
2	Randbedingungen	7
2.1	Technische Randbedingungen	7
2.2	Organisatorische Randbedingungen	7
2.3	Konventionen	7
3	Kontextabgrenzung	9
3.1	Fachlicher Kontext	9
3.2	Technischer Kontext	9
4	Lösungsstrategie	11
4.1	Technologie-Entscheidungen	11
4.2	Top-Level-Zerlegung	11
4.3	Wesentliche Designprinzipien	11
4.4	Erreichung der Qualitätsziele	12
5	Bausteinsicht	13
5.1	Whitebox Gesamtsystem	13
5.1.1	Enthaltene Bausteine	13
5.1.2	Wichtige Schnittstellen	14
5.2	Ebene 2 — Detailzerlegung e-gov-framework (Kern)	14
5.2.1	Persistence (ch.ti8m.egov.framework.persistence)	14
5.2.2	IAM (ch.ti8m.egov.framework.iam)	14
5.2.3	Command & Validation (ch.ti8m.egov.framework.validation)	15
5.2.4	Exception Handling & Context (ch.ti8m.egov.framework.exceptionhandling)	15
5.3	Ebene 2 — Detailzerlegung e-gov-ai-rule-generation	15
6	Laufzeitsicht	16
6.1	Szenario 1 — Schreiboperation via Command Pattern	16
6.2	Szenario 2 — Gefilterte Listenabfrage mit Row-based Security	16
6.3	Szenario 3 — Authentication-Flow (Vorgelagertes IAM)	17
6.4	Szenario 4 — AI-gestützte Regelgenerierung	17
7	Verteilungssicht	18
7.1	Motivation	18
7.2	Infrastruktur Ebene 1	18
7.2.1	Software-Infrastruktur-Zuordnung	19
7.3	Sicherheitszonen	19
8	Querschnittliche Konzepte	20
8.1	Sicherheit und Autorisierung	20
8.1.1	Authentifizierung (vorgelagert)	20
8.1.2	Row-based Security	20
8.2	Command-Pattern und Audit-Trail	21
8.2.1	Status-Lifecycle	21

8.2.2	ExecutionPlan	21
8.2.3	Executor-Varianten	21
8.3	Fehlerbehandlung	22
8.3.1	Exception-Hierarchie	22
8.3.2	Einheitliche Fehlerantwort	22
8.3.3	Fehler-Code-Schema	22
8.4	Persistenz	23
8.4.1	Repository-Pattern	23
8.4.2	Filter-DSL	23
8.4.3	Multi-Datenbankunterstützung	23
8.4.4	Datenbankmigrationen	23
8.5	Konfiguration	23
8.6	Logging	24
8.7	Internationalisierung	24
8.7.1	Datenstruktur	24
8.7.2	Sprachauswahl	25
8.8	ID-Generierung (LUID)	25
8.8.1	Algorithmus (LUIDGenerator)	25
8.9	Testbarkeit	25
8.9.1	Test-Pyramide	25
8.9.2	Testcontainers-Konfiguration	26
9	Architekturentscheidungen	27
9.1	ADR-001: Framework als eingebettete Java-Bibliothek, nicht als eigenständiger Service	27
9.1.1	Kontext	27
9.1.2	Entscheidung	27
9.1.3	Konsequenzen	28
9.2	ADR-002: Native JDBC für die Query-Schicht statt JPA Criteria API	28
9.2.1	Kontext	28
9.2.2	Entscheidung	28
9.2.3	Konsequenzen	28
9.3	ADR-003: Command-Pattern für alle Schreiboperationen	29
9.3.1	Kontext	29
9.3.2	Entscheidung	29
9.3.3	Konsequenzen	29
9.4	ADR-004: Row-based Security als opt-out statt opt-in	29
9.4.1	Kontext	30
9.4.2	Entscheidung	30
9.4.3	Konsequenzen	30
9.5	ADR-005: Vorgelagertes IAM — kein eigener Authentifizierungsstack	30
9.5.1	Kontext	30
9.5.2	Entscheidung	31
9.5.3	Konsequenzen	31
9.6	ADR-006: Unterstützung von PostgreSQL und SQL Server parallel	31
9.6.1	Kontext	31
9.6.2	Entscheidung	32
9.6.3	Konsequenzen	32
9.7	ADR-007: Spring AI + Azure OpenAI für LLM-gestützte Regelgenerierung	32
9.7.1	Kontext	32
9.7.2	Entscheidung	32
9.7.3	Konsequenzen	33
9.8	ADR-008: LUID als cluster-sichere ID-Generierungsstrategie	33
9.8.1	Kontext	33
9.8.2	Entscheidung	33
9.8.3	Konsequenzen	33

10 Qualitätsanforderungen	34
10.1 Qualitätsbaum	34
10.2 Qualitätsszenarien	34
10.2.1 QS-01: Neue CRUD-Domäne in kurzer Zeit	34
10.2.2 QS-02: Breaking Changes vor dem Upgrade erkennbar	35
10.2.3 QS-03: Gefilterte Listenabfrage unter 500 ms	35
10.2.4 QS-04: Kein Benutzer sieht unberechtigt Datensätze	36
10.2.5 QS-05: Jede Operation nachvollziehbar	36
10.2.6 QS-06: Pod-Ausfall ohne Datenverlust	37
10.2.7 QS-07: Datenbankwechsel ohne Code-Änderung	37
10.2.8 QS-08: Integrationstests ohne produktive Infrastruktur	37
10.2.9 QS-09: Stammdatenkonsistenz über Applikationsgrenzen	38
10.3 Mengengerüst	38
11 Risiken und technische Schulden	39
11.1 Risiken	39
12 Glossar	41
12.1 Framework-Konzepte	41
12.2 Systemkomponenten und externe Systeme	42
12.3 Abkürzungen	43

1 Einführung und Ziele

1.1 Aufgabenstellung

Das e-gov-framework ist eine Java-Bibliothek (kein eigenständiges System), die Schweizer Behörden und Ämter dabei unterstützt, E-Government-Fachapplikationen effizient und sicher zu entwickeln. Es stellt technische Grundfunktionen bereit, die in allen E-Gov-Projekten wiederkehren: regelbasierte Autorisierung auf Zeilen- und Feldebene, ein generisches Persistence-Layer mit dynamischer Filtersprache, Stammdatenverwaltung (MDM) sowie ein Command-basiertes Verarbeitungsmodell für nachvollziehbare Schreiboperationen.

Das Framework wird von ti&m AG entwickelt und über ein Maven-Repository (Artifactory) bereitgestellt. Behörden beziehen es als Abhängigkeit und bauen darauf ihre spezifischen Fachapplikationen auf.

1.2 Qualitätsziele

Priorität	Qualitätsziel	Motivation / Szenario
1	Wartbarkeit / Developer Experience	Ein Entwicklerteam einer Behörde soll innerhalb eines Tages eine neue CRUD-Domäne (Entity, Repository, Command, REST-Endpoint) vollständig integrieren können
2	Performance der Query-Schicht	Eine gefilterte, paginierte Liste von 10 000 Entitäten muss inklusive Permission-Filter in unter 500 ms geliefert werden
3	Row-based Security	Jede Datenbankabfrage wird automatisch um rollenbasierte WHERE-Klauseln erweitert — ein Benutzer sieht niemals Datensätze, für die er keine Berechtigung besitzt
4	Stammdatenkonsistenz	Das zentrale MDM-Modul stellt sicher, dass alle Fachapplikationen identische, mehrsprachige Codelisten verwenden
5	Nachvollziehbarkeit / Audit-Trail	Jede fachliche Schreiboperation ist mit Benutzer, Zeitstempel und Status in der Command-Tabelle protokolliert (Compliance-Anforderung der Behörden)
6	Zuverlässigkeit	Das Framework verhält sich bei Pod-Ausfällen und Startfehlern vorhersehbar — Datenverlust und stiller Misserfolg werden verhindert
7	Portabilität / Datenbankflexibilität	Ein Datenbankwechsel zwischen PostgreSQL und SQL Server erfordert keine Code-Änderungen in consuming Apps
8	Betreibbarkeit	Fehlkonfigurationen werden beim Start erkannt und gemeldet, bevor Traffic auf den Pod geleitet wird
9	Testbarkeit	Integrationstests laufen ohne produktive Infrastruktur, sodass Entwickler und CI-Pipelines schnell Feedback erhalten

1.3 Stakeholder

Rolle	Erwartung
Framework-Team (ti&m)	Wartbare, gut testbare Codebasis; klare Release-Prozesse via Semantic Release
Entwicklerteams der Behörden	Schneller Einstieg, geringe Boilerplate, stabile API zwischen Minor-Versionen
Behörden / Ämter (Betreiber)	Compliance mit Sicherheitsvorgaben, stabiler Betrieb, klar dokumentierte Konfiguration
Endnutzer (Sachbearbeiter)	Schnelle Antwortzeiten, korrekte Zugriffskontrolle auf Daten
Sicherheitsverantwortliche	Nachvollziehbarer Audit-Trail (Command-Tabelle), row-based Security

2 Randbedingungen

2.1 Technische Randbedingungen

Randbedingung	Erläuterung
Java 21	LTS-Version; Pflicht für alle Framework-Module und consuming Applications
Spring Boot 3.5	Basis-Framework; alle Auto-Konfigurationen setzen Spring Boot 3.x voraus
Maven	Build-Tool; Framework wird als Maven-BOM + einzelne JARs via Artifactory distribuiert
PostgreSQL oder SQL Server	Zwei unterstützte Datenbanktypen; PostgreSQL empfohlen, SQL Server für Kunden mit Microsoft-Vorgaben
Flyway	Datenbank-Migrationen; Framework liefert eigene Flyway-Skripte; Applikationen ergänzen eigene
Kubernetes (On-Premise)	Deployment-Plattform aller produktiven Installationen. Ausnahme: Das optionale Modul e-gov-ai-rule-generation kommuniziert mit Azure OpenAI (Cloud-Egress erforderlich); Behörden, die dieses Modul einsetzen, benötigen eine entsprechende Netzwerk-Freigabe (→ ADR-007)
Docker	Container-Laufzeitumgebung; lokale Entwicklung via docker-compose (PostgreSQL, SQL Server)
Azure AD + F5	Vorgelagertes Identity-System; Framework erhält userId via EGOV_TOKEN-Cookie, führt selbst keine Authentication durch

2.2 Organisatorische Randbedingungen

Randbedingung	Erläuterung
Entwicklung durch ti&m AG	Das Framework ist ein internes ti&m-Produkt; Issues und Releases werden intern gesteuert
Hosting durch Kundenwahl	Jede Behörde wählt ihren eigenen Hosting-Provider; das Framework muss auf beliebigen On-Premise-Kubernetes-Clustern betreibbar sein. Das optionale KI-Modul setzt zusätzlich Azure-OpenAI-Konnektivität voraus und ist daher nur für Behörden mit entsprechender Cloud-Freigabe geeignet (→ ADR-007)
Semantic Versioning	Releases folgen SemVer via Conventional Commits; CHANGELOG wird automatisch generiert
ArgoCD für Deployments	Applikationen werden über GitOps-Workflows (ArgoCD) auf Kubernetes ausgerollt
Mehrere aktive Produkktivsysteme	Bei Kunden der ti&m; Breaking Changes erfordern koordinierte Migrationen

2.3 Konventionen

Konvention	Erläuterung
Conventional Commits	Commit-Messages nach feat:, fix:, chore:-Schema; steuert automatische Versionierung
Deutsch als Dokumentationssprache	Fachliche Konzepte und Kommentare auf Deutsch; Code auf Englisch
egov.* Konfigurations-Präfix	Alle Framework-Properties gruppieren sich unter dem Namespace egov. in application.yml
SNAKE_CASE oder CAMEL_CASE	Datenbankspaltennamen-Konvention ist pro Deployment konfigurierbar; Default: CAMEL_CASE
OpenAPI via SpringDoc	Alle REST-Endpunkte werden automatisch über /swagger-ui.html dokumentiert

3 Kontextabgrenzung

3.1 Fachlicher Kontext

Das e-gov-framework selbst hat keine fachliche Benutzeroberfläche. Es ist eine Bibliothek, die von Fachapplikationen verwendet wird.

Partner	Eingehende Daten	Ausgehende Daten
Sachbearbeiter (Behörde)	Formulardaten, Filter- und Suchanfragen, CRUD-Operationen	Gefilterte Datenlisten (nur berechnigte Zeilen), Validierungsfehler, Statusmeldungen
Behörden-IAM (Azure AD + F5)	EGOV_TOKEN-Cookie mit userId	—
Stammdaten-Konsumenten (MDM)	Abfragen auf Codelisten (Ortschaften, Sprachen, Anreden etc.)	Mehrsprachige Code-Listen

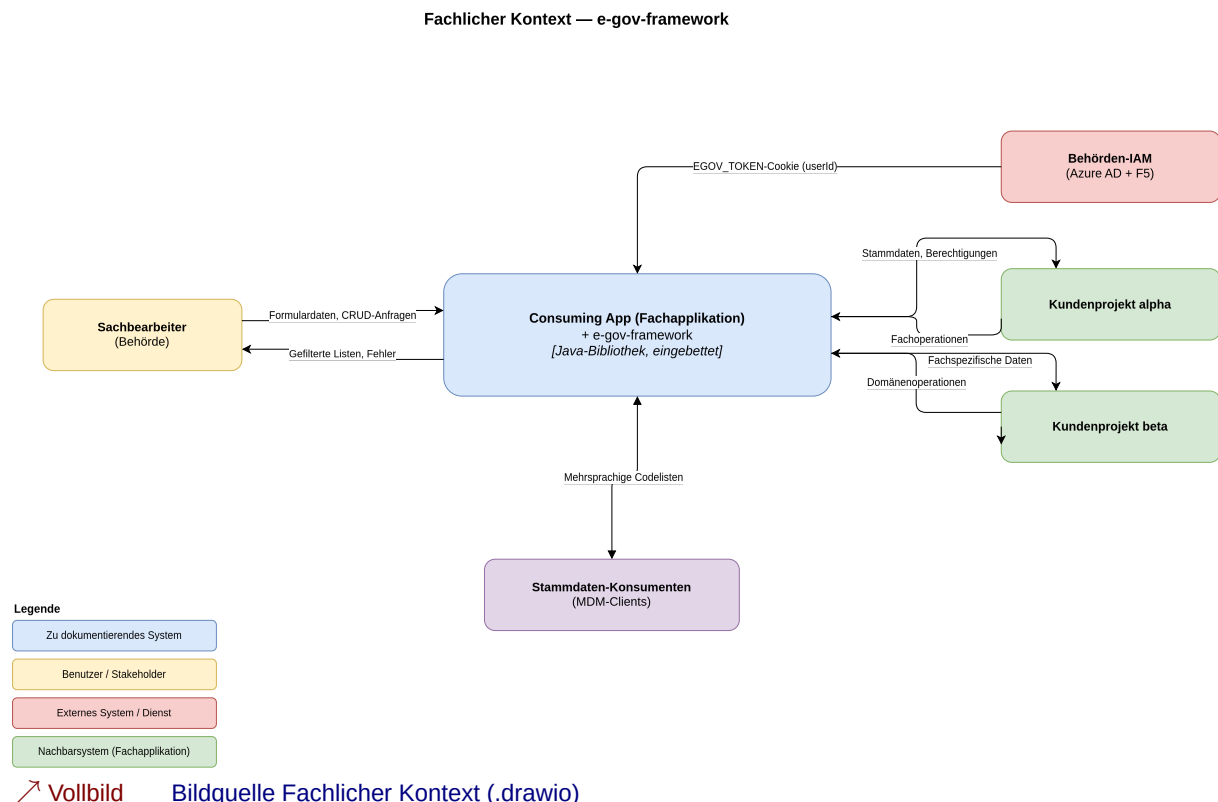


Abbildung 3.1: Fachlicher Kontext

3.2 Technischer Kontext

Schnittstelle	Protokoll	Richtung	Bemerkung
Azure AD + F5 Firewall	HTTPS / Cookie	→ Framework	Setzt EGOV_TOKEN-Cookie nach Login; Framework extrahiert userId über UserIdProvider-Interface
PostgreSQL	JDBC	↔	Primäre Produktionsdatenbank; Framework liefert Flyway-DDL-Skripte
SQL Server	JDBC	↔	Alternative für Kunden mit Microsoft-Stack
ZooKeeper	TCP (Port 2181)	↔	Optional; für verteilte Command-Ausführung über mehrere Instanzen; via Apache Curator
Azure OpenAI (gpt-4o)	HTTPS / REST	→	Nur e-gov-ai-rule-generation-Modul; generiert Validierungsregeln aus natürlicher Sprache
Artifactory (bin.ti8m.ch)	HTTPS / Maven	←	Framework-JARs werden von consuming Apps via Maven bezogen
ArgoCD	Kubernetes API	→	Deployment-Steuerung der consuming Apps; kein direkter Framework-Kontakt

4 Lösungsstrategie

4.1 Technologie-Entscheidungen

Bereich	Entscheidung	Begründung
Persistence	JPA/Hibernate + nativer JDBC für Queries	JPA für Entity-Mapping; native SQL im QueryBuilder für volle Kontrolle über Permission-Filter und dynamische WHERE-Klauseln
Datenbankmigrationen	Flyway	Versionierte, reproduzierbare Schema-Migrationen; Framework liefert eigene Skripte mit
Multi-DB	PostgreSQL + SQL Server	Behörden haben unterschiedliche Infrastrukturvorgaben; beide Dialekte werden unterstützt
Build & Distribution	Maven BOM + Artifactory	BOM ermöglicht konsistentes Dependency-Management; Artifactory als internes Registry
AI-Integration	Spring AI + Azure OpenAI	Regelgenerierung aus Freitext; Spring AI als Abstraktionsschicht für spätere LLM-Wechselbarkeit
Service Discovery	Apache Curator / ZooKeeper	Optionale verteilte Command-Ausführung über Instanzen hinweg
ID-Generierung	LUID / DB-Sequence / ULID (e-gov-luid)	Sortierbare, gap-tolerante IDs je nach Anforderung der Applikation

4.2 Top-Level-Zerlegung

Das Framework ist als **Maven-Bibliothek** strukturiert — es läuft nicht selbst, sondern wird in Fachapplikationen eingebettet. Die Zerlegung folgt dem Prinzip **Kern + optionale Erweiterungsmodule**:

e-gov-bom	← Dependency-Versionssteuerung (kein Code)
e-gov-framework Handling	← Pflicht-Kern: Persistence, IAM, Command-Pattern, Exception-
e-gov-luid	← Optional: alternative ID-Generierungsstrategien
e-gov-mdm	← Optional: Stammdatenverwaltung (Codelisten, Vokabulare)
e-gov-mdm-api	← Optional: REST-API-Layer für MDM
e-gov-ai-rule-generation	← Optional: LLM-gestützte Regelgenerierung
e-gov-framework-demo	← Referenzimplementierung (nicht für Produktion)

4.3 Wesentliche Designprinzipien

- **Convention over Configuration** — Spring Boot Auto-Konfiguration; consuming Apps benötigen nur wenige egov.*-Properties
- **Command Pattern für alle Schreiboperationen** — jede fachliche Aktion wird als Command-Entity persistiert; ermöglicht Audit-Trail und nachvollziehbaren Rollback

- **Deklarative Autorisierung via Annotationen** — @Ruleset, @AllowedForRoles, @Validation auf DTOs; Laufzeit-Auswertung durch ValidationEngine
- **Row-based Security by Default** — QueryBuilder fügt automatisch Permission-Filter ein; kein explizites opt-in nötig
- **Eigene Filter-DSL** — ermöglicht dynamische, ausdrucksstarke WHERE-Klauseln aus REST-Anfragen ohne SQL-Injection-Risiko
- **Soft-Delete** — Archivierung statt physischem Löschen via ArchivedModifiableEntity

4.4 Erreichung der Qualitätsziele

Qualitätsziel	Massnahme
Wartbarkeit	BOM, Convention over Configuration, generische Basisklassen (BaseRepositoryImpl, BaseExecutor, BaseAction) minimieren Wiederholung in consuming Apps
Performance Query-Schicht	Nativer JDBC im QueryBuilder statt JPA-Criteria-API; Permission-Filter direkt in SQL; konfigurierbare Paginierung
Row-based Security	PermissionBuilder erweitert jede SELECT-Abfrage automatisch um rollenbasierte WHERE-Bedingungen; keine manuelle Filterung nötig
Stammdatenkonsistenz	Zentrales MDM-Modul mit mehrsprachiger Unterstützung; alle Apps greifen auf denselben Vokabular-Store zu

5 Bausteinsicht

5.1 Whitebox Gesamtsystem

Das e-gov-framework besteht aus einem Pflicht-Kernmodul und vier optionalen Erweiterungsmodulen. Fachapplikationen beziehen die Module als Maven-Dependencies über das BOM.

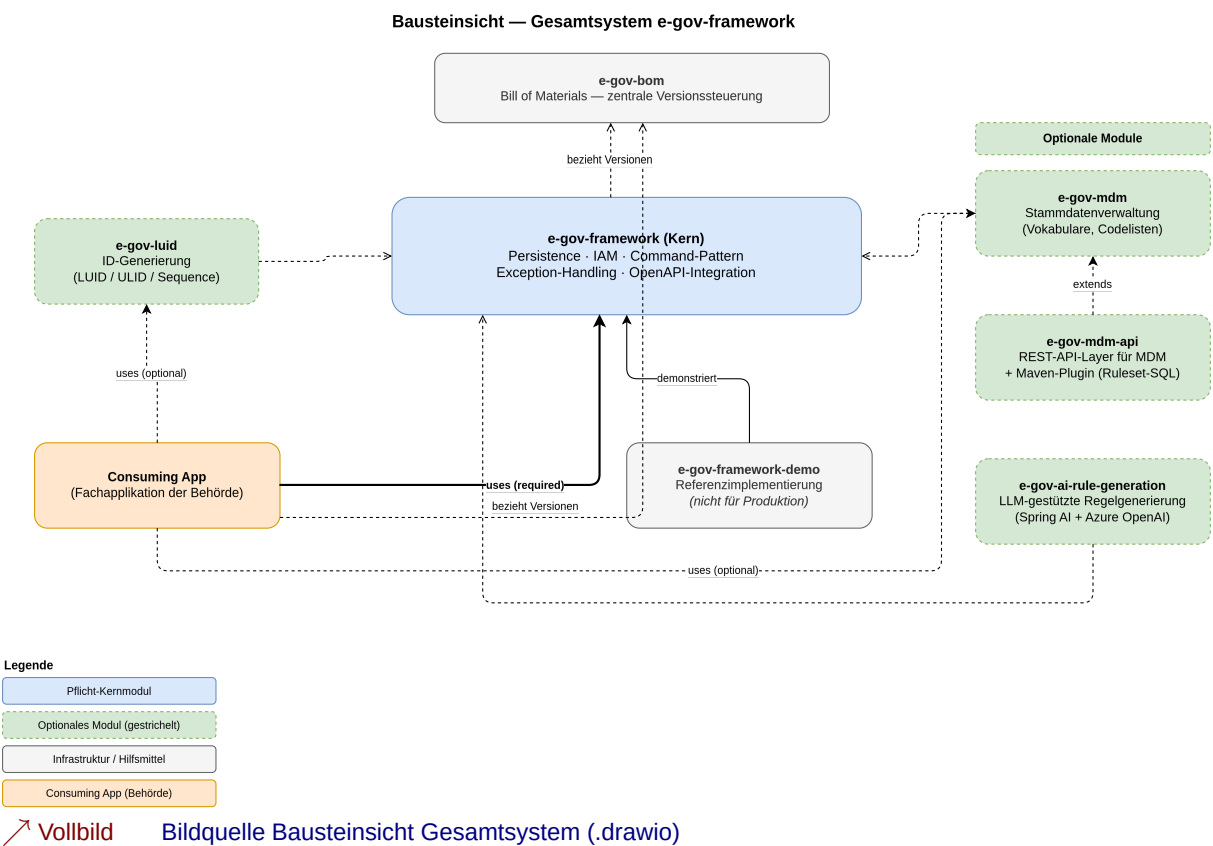


Abbildung 5.1: Bausteinsicht Gesamtsystem

5.1.1 Enthaltene Bausteine

Baustein	Verantwortung
e-gov-bom	Bill of Materials — zentrale Versionssteuerung aller Framework-Abhängigkeiten; kein ausführbarer Code
e-gov-framework	Kern: Persistence-Abstraktion, IAM (Rollen/RuleSets), Command-Pattern, Exception-Handling, OpenAPI-Integration
e-gov-mdm	Stammdatenverwaltung: Vokabulare (Codelisten), mehrsprachige Codes, erweiterbare Felddefinitionen
e-gov-mdm-api	REST-API-Layer für MDM; OpenAPI-Dokumentation; Maven-Plugin zur Ruleset-SQL-Generierung

Baustein	Verantwortung
e-gov-ai-rule-generation	LLM-gestützte Generierung von Validierungsregeln aus natürlicher Sprache (Azure OpenAI / gpt-4o)
e-gov-luid	Alternative ID-Generierungsstrategien: LUID (sortierbar, cluster-sicher), DB-Sequence, ULID
e-gov-framework-demo	Referenzimplementierung; zeigt korrekten Einsatz aller Module

5.1.2 Wichtige Schnittstellen

Schnittstelle	Beschreibung
<code>BaseRepositoryImpl<T></code>	Abstrakte Basisklasse; consuming Apps erweitern sie für jede Domänen-Entity
<code>BaseExecutor</code>	Abstrakte Basis für Command-Executor; kapselt Validation + Persistierung
<code>UserIdProvider</code>	Interface; consuming Apps implementieren es, um <code>userId</code> aus dem <code>EGOV_TOKEN-Cookie</code> zu extrahieren
<code>DomainCommandFactory</code>	Abstrakte Fabrik; consuming Apps erzeugen domänenspezifische Commands
<code>@Ruleset</code> / <code>@Validation</code>	Annotationen auf DTOs; deklarieren Autorisierungs- und Validierungsregeln
<code>egov.* Properties</code>	Konfigurationspunkte in <code>application.yml</code> der consuming App

5.2 Ebene 2 — Detailzerlegung e-gov-framework (Kern)

Der Kern gliedert sich in vier interne Pakete.

5.2.1 Persistence (`ch.ti8m.egov.framework.persistence`)

Baustein	Verantwortung
<code>BaseRepositoryImpl</code> <code>QueryBuilder</code>	Generische CRUD-Operationen; delegiert Query-Bau an <code>QueryBuilder</code> Baut native SQL-Abfragen aus Filter-DSL, Sortierung, Paginierung und Permission-Filter
<code>FilterParser</code> / <code>Tokenizer</code>	Parsen den Filter-DSL-String in einen Abstract Syntax Tree
<code>PermissionBuilder</code>	Ergänzt jede SELECT-Abfrage um rollenbasierte WHERE-Klauseln
<code>ModifiableEntity</code>	Basis-JPA-Entity mit Audit-Feldern (<code>createdBy</code> , <code>modifiedBy</code> , Zeitstempel)
<code>DatabaseConfigurationService</code>	Liest <code>egov.persistence.database.type</code> und <code>naming.style</code> ; konfiguriert Dialekt

5.2.2 IAM (`ch.ti8m.egov.framework.iam`)

Baustein	Verantwortung
<code>RuleSetService</code>	Ermittelt den passenden RuleSet für eine Action + Benutzerrolle
<code>PermissionService</code>	Prüft ob ein Benutzer eine Operation (READ/WRITE/DELETE) auf einer Entity ausführen darf
<code>RoleService</code>	Verwaltet Rollen und Rollenmitgliedschaften
<code>IAMController</code>	REST-Endpoint <code>GET /iam/v1/permissions</code>
<code>RuleSet-Entities</code>	JPA-Entities in Tabellen <code>egov_iam_rule_set</code> , <code>egov_iam_condition</code> , <code>egov_iam_role*</code>

5.2.3 Command & Validation (ch.ti8m.egov.framework.validation)

Baustein	Verantwortung
Command	JPA-Entity; persistiert jede fachliche Aktion (Tabelle egov_cmd_command) mit Status-Lifecycle (INITIATED → VALIDATED → EXECUTED / FAILED)
DomainCommandFactory	Erzeugt und initialisiert Commands; setzt Audit-Informationen
BaseExecutor	Orchestriert Validation (via ValidationEngine) → Ausführung → Persistierung
ValidationEngine	Wertet RuleSet-Validierungsregeln rekursiv gegen DTO-Felder aus
NonTxExecutor	Führt Commands ausserhalb einer Spring-Transaktion aus

5.2.4 Exception Handling & Context (ch.ti8m.egov.framework.exceptionhandling)

Baustein	Verantwortung
GenericExceptionHandler	@ControllerAdvice; wandelt alle EGovException-Subklassen in strukturierte HTTP-Fehlerantworten um
DataHolder	ThreadLocal-Kontext; hält userId für den aktuellen Request
ContextInterceptor	HTTP-Interceptor; extrahiert userId aus EG0V_TOKEN und befüllt DataHolder
EGovException-Hierarchie	ActionNotAllowedException, NoPermissionForEntityException, UnauthorizedException, SystemException

5.3 Ebene 2 — Detailzerlegung e-gov-ai-rule-generation

Das KI-Modul ist als optionales Erweiterungsmodul konzipiert und enthält drei Bausteine, die den Workflow von der Freitext-Eingabe bis zur persistierten Validierungsregel abbilden.

Baustein	Verantwortung
RuleModificationController	REST-Endpoint (POST); nimmt Freitext-Beschreibung entgegen und orchestriert Generierung und Persistierung der neuen Regel
RuleGenerator	Baut den Prompt aus Freitext + Regelgrammatik und ruft Azure OpenAI (gpt-4o) über Spring AI auf
RuleTranslator	Konvertiert die LLM-Antwort in das Framework-Regelformat und persistiert die Regel via RuleSetService in egov_iam_rule_set

Externe Abhängigkeit: RuleGenerator kommuniziert mit Azure OpenAI über HTTPS (Cloud-Egress); die übrigen Bausteine kommunizieren ausschliesslich mit dem e-gov-framework-Kern (→ Szenario 4 in Sektion 6).

6 Laufzeitsicht

6.1 Szenario 1 — Schreiboperation via Command Pattern

Ablauf einer typischen Schreiboperation (z.B. Anlegen einer neuen Entität) in einer Fachapplikation:

```
Sachbearbeiter → REST-Controller (consuming App)
  → DomainCommandFactory.getCommand(action, dto, parameters)
  → Command.status = INITIATED (persistiert in egov_cmd_command)
  → BaseExecutor.execute(command)
    → RuleSetService.getApplicableRuleSet(command)      ← DB-Abfrage egov_iam_*
    → ValidationEngine.validate(ruleSet, dto, action)
      → Validierungsregeln rekursiv ausgewertet → ValidationTriple
  → [FAILED] → Command.status = HARD_VALIDATION_FAILED, ResponseExceptionBody zurückgegeben
  → [OK]      → Command.status = VALIDATED
                → BaseRepositoryImpl.saveWithTx(entity)
                  → Command.status = EXECUTED
                  → HTTP 200 / 201
```

Beteiligte Bausteine: REST-Controller (App), DomainCommandFactory, BaseExecutor, ValidationEngine, RuleSetService, BaseRepositoryImpl

6.2 Szenario 2 — Gefilterte Listenabfrage mit Row-based Security

Ablauf einer paginierten GET-Anfrage mit Filterausdruck:

```
Sachbearbeiter → REST-Controller (consuming App)
  → BaseRepositoryImpl.find(parameters)
    → QueryBuilder.buildBaseQuery(parameters, entity, config)
      → FilterParser.parse(parameters.filter)      ← Filter-DSL → AST
    → PermissionBuilder.append(query, userId)      ← Rollen-WHERE aus egov_iam_permission_definition
      → Sortierung + LIMIT/OFFSET angefügt
    → Native SQL via JDBC ausgeführt
    → PaginatedResult<T> zurückgegeben
  → HTTP 200 mit Datenliste (ausschliesslich berechnete Zeilen)
```

Beteiligte Bausteine: REST-Controller, BaseRepositoryImpl, QueryBuilder, FilterParser, PermissionBuilder, DataHolder

6.3 Szenario 3 — Authentication-Flow (Vorgelagertes IAM)

Sachbearbeiter → Azure AD (Login)

- F5-Firewall setzt EGOV_TOKEN-Cookie
- HTTP-Request mit Cookie → ContextInterceptor (Framework)
 - UserIdProvider.getUserId(request) ← Implementierung in consuming App
 - DataHolder.setUserId(userId) ← ThreadLocal für Request-Dauer
- Alle nachgelagerten Permission-Checks lesen userId aus DataHolder

Beteiligte Bausteine: Azure AD (extern), F5-Firewall (extern), ContextInterceptor, UserIdProvider (consuming App), DataHolder

6.4 Szenario 4 — AI-gestützte Regelgenerierung

Ablauf der Generierung einer neuen Validierungsregel aus natürlicher Sprache:

Framework-Entwickler / Admin → RuleModificationController (POST)

- RuleGenerator.generate(freitextBeschreibung)
 - Azure OpenAI API aufgerufen
 - Prompt enthält Grammatik der erlaubten Validierungsfunktionen
- RuleTranslator.translate(llmResponse)
 - JSON → Framework-Validierungsregel-Format
- Regel in egov_iam_rule_set persistiert
- HTTP 200 mit generierter Regel

Beteiligte Bausteine: RuleModificationController, RuleGenerator, RuleTranslator, Azure OpenAI (extern), RuleSetService

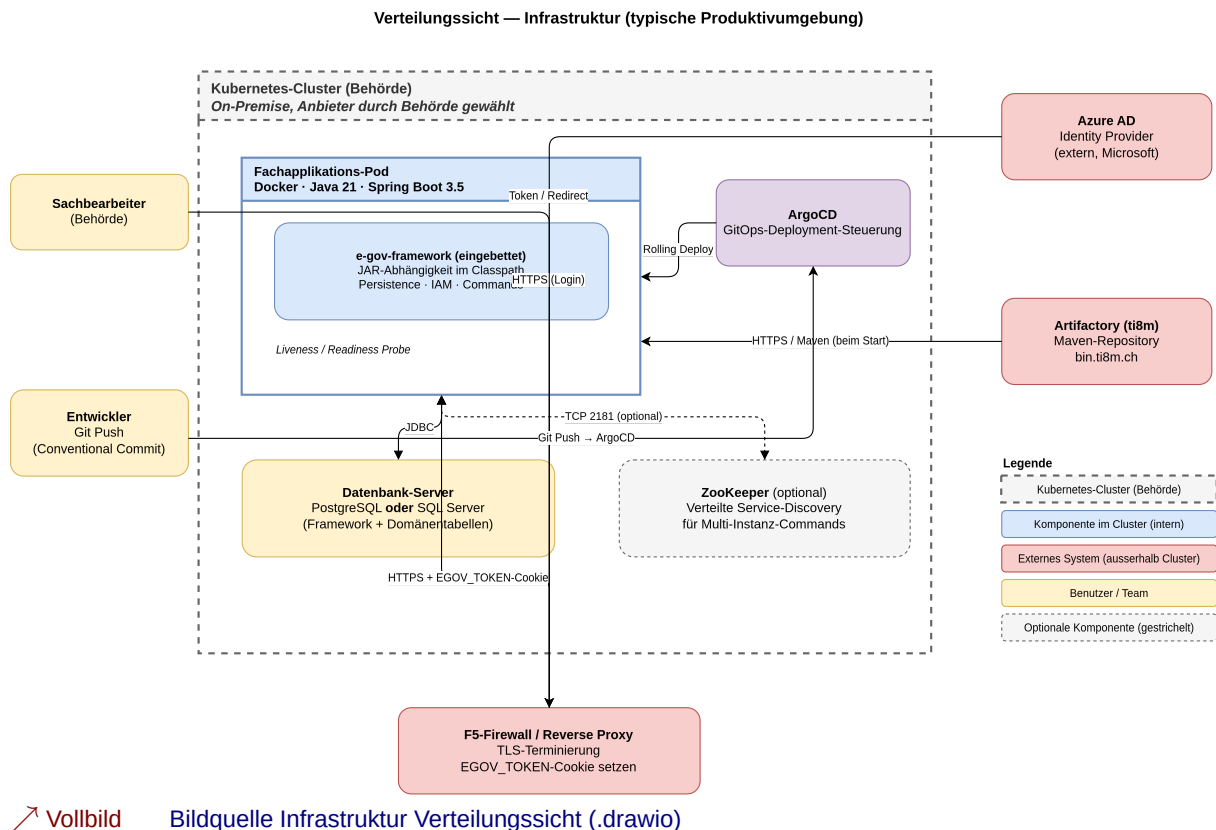
7 Verteilungssicht

7.1 Motivation

Das e-gov-framework wird vollständig On-Premise auf kundeneigenen Kubernetes-Clustern betrieben. Diese Entscheidung folgt aus behördlichen Anforderungen an Datensouveränität, Compliance und Netzwerkisolation. Alle Kernmodule benötigen ausschliesslich Infrastruktur innerhalb des Clusters (→ Randbedingungen Sektion 2). Das optionale KI-Modul e-gov-ai-rule-generation bildet die einzige Ausnahme: Es kommuniziert mit Azure OpenAI über HTTPS und erfordert daher eine explizite Netzwerk-Freigabe durch die Behörde (→ ADR-007).

7.2 Infrastruktur Ebene 1

Typische Produktivumgebung einer Behörde:



↗ **Vollbild** Bildquelle Infrastruktur Verteilungssicht (.drawio)

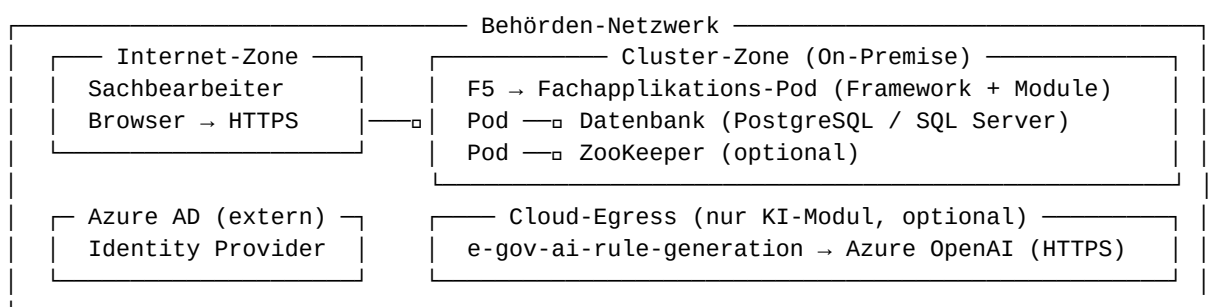
Abbildung 7.1: Infrastruktur Verteilungssicht

Knoten	Rolle	Technologie
Kubernetes-Cluster	Laufzeitumgebung für alle Applikations-Pods	Kubernetes On-Premise; Anbieter durch Behörde gewählt
Fachapplikations-Pod	Laufzeitumgebung der consuming App (Framework eingebettet als JAR)	Docker-Container, Java 21, Spring Boot
Datenbank-Server	Persistenz für Framework- und Domänentabellen	PostgreSQL oder SQL Server
ZooKeeper (optional)	Verteilte Service Discovery für Multi-Instanz-Deployments	Apache ZooKeeper 3.x
F5-Firewall / Reverse Proxy	TLS-Terminierung, Cookie-Setzen (EGOV_TOKEN) nach Login	F5 BIG-IP
Azure AD	Identity Provider, Authentication	Microsoft Azure AD (extern)
ArgoCD	GitOps-Deployment-Steuerung	ArgoCD auf demselben oder separatem Cluster
Artifactory (ti&m)	Maven-Repository für Framework-JARs	bin.ti8m.ch (ti8m-betrieben)

7.2.1 Software-Infrastruktur-Zuordnung

Framework-Modul	Laufzeitknoten	Netzwerk-Zone
e-gov-framework (Kern)	Fachapplikations-Pod	Cluster-intern
e-gov-mdm	Fachapplikations-Pod	Cluster-intern
e-gov-mdm-api	Fachapplikations-Pod	Cluster-intern
e-gov-luid	Fachapplikations-Pod	Cluster-intern
e-gov-bom	Build-Zeit (kein Laufzeittartefakt)	—
e-gov-framework-demo	DEV/INT-Umgebung (nicht Produktion)	Cluster-intern
e-gov-ai-rule-generation	Fachapplikations-Pod	Cluster-intern + Cloud-Egress (Azure OpenAI)

7.3 Sicherheitszonen



Sicherheitszonen: - **Internet-Zone:** Eingehender HTTPS-Traffic der Sachbearbeiter; TLS-Terminierung durch die F5-Firewall; EGOV_TOKEN-Cookie wird gesetzt - **Cluster-Zone (On-Premise):** Alle Framework-Kernmodule und Datenbankserver; kein direkter Internetzugang; Kommunikation nur Pod-zu-Pod und Pod-zu-DB - **Cloud-Egress (optional):** Ausschliesslich e-gov-ai-rule-generation; erfordert explizite Firewall-Freigabe durch die Behörde (→ ADR-007)

8 Querschnittliche Konzepte

Dieses Kapitel beschreibt technische Konzepte, die mehrere Bausteine des e-gov-framework betreffen und systemweit einheitlich umgesetzt sind.

8.1 Sicherheit und Autorisierung

8.1.1 Authentifizierung (vorgelagert)

Das Framework führt keine eigene Authentifizierung durch. Die Identität des Nutzers wird durch die vorgelagerte Infrastruktur festgestellt. Die F5-Firewall setzt nach erfolgreichem Login einen EGOV_TOKEN-Cookie. Der ContextInterceptor des Frameworks extrahiert daraus die userId über das UserIdProvider-Interface, das jede consuming App implementiert, und speichert sie für die Dauer des Requests im DataHolder-ThreadLocal.

HTTP-Request

- ContextInterceptor.preHandle()
 - UserIdProvider.getUserId(request) ← consuming-App-Implementierung
 - DataHolder.setUserId(userId) ← ThreadLocal
- alle nachgelagerten Komponenten lesen userId über DataHolder.getUserId()
- ContextInterceptor.afterCompletion()
 - DataHolder.clear() ← ThreadLocal aufräumen

8.1.2 Row-based Security

Jede Datenbankabfrage wird automatisch um rollenbasierte WHERE-Klauseln erweitert. Kein Code in der consuming App muss explizit filtern — die Sicherheit greift auf Infrastruktur-ebene im QueryBuilder.

Datenmodell (Flyway DDL):

Tabelle	Zweck
egov_iam_role	Rollendefinitionen
egov_iam_role_membership	Benutzer-Rollen-Zuordnungen (mit valid_from/valid_to)
egov_iam_permission_definition	Rollenspezifische SQL-Filterbedingungen pro Entity und Operation
egov_iam_rule_set	Validierungsregelsets pro Action
egov_iam_condition	Bedingungen innerhalb eines RuleSets

Ablauf:

```
BaseRepositoryImpl.find(parameters)
```

- `PermissionBuilder.getPermissions(userId, entityClass, operation)`
- `PermissionService.getAllowedRecordsQuery(userId, operation, clazz)`
 - Rollenermittlung aus `egov_iam_role_membership` (`userId`, aktuelles Datum)
 - SQL-Filterausdruck aus `egov_iam_permission_definition` (`role`, `entity`, `operation`)
 - Rückgabe: `AllowedRecordsQuery` (Query-String + Parameter-Werte)
- `ParametrizedQuery` wird als `WHERE`-Klausel in native SQL integriert

8.2 Command-Pattern und Audit-Trail

Alle fachlichen Operationen werden als Command-JPA-Entity in der Tabelle `egov_cmd_command` persistiert. Dadurch entsteht ein lückenloses Audit-Log aller Aktionen.

8.2.1 Status-Lifecycle

INITIALIZED

- `RULESET_SELECTED` (nach `RuleSet`-Auflösung)
- `VALIDATED` (nach Validierung)
- `COMPLETED`
- `EXECUTED` (Endzustand: Erfolg)
- `HARD_VALIDATION_FAILED` (Endzustand: Fehler)

Beide Felder `status` und `executionPlan` sind als `@Enumerated(EnumType.STRING)` gemappt.

8.2.2 ExecutionPlan

Der `ExecutionPlan` steuert, ob und in welcher Reihenfolge Validierung und Ausführung stattfinden:

Wert	Verhalten
<code>VALIDATE_BEFORE_EXECUTION</code>	Bricht bei Validierungsfehler ab (Standard)
<code>VALIDATE_BEFORE_EXECUTING_ANYWAY</code>	Validiert, führt aber immer aus
<code>VALIDATE_AFTER_EXECUTION</code>	Führt zuerst aus, validiert danach
<code>DONT_VALIDATE</code>	Keine Validierung
<code>DONT_EXECUTE</code>	Keine Ausführung (z.B. Dry-Run)

8.2.3 Executor-Varianten

Interface	Merkmal	Verwendung
<code>Executor</code>	<code>@Transactional</code> auf <code>execute()</code>	Standard-Schreiboperationen
<code>NonTxExecutor</code>	kein <code>@Transactional</code>	Operationen, die ausserhalb einer Spring-Transaktion laufen müssen (bekanntes Risiko R-2)
<code>RemoteCommandExecutor</code>	Ausführung über ZooKeeper	Verteilte Commands über mehrere Instanzen
<code>BatchUpdateExecutor<T></code>	Batch-Verarbeitung mit Progress-Logging	Massenoperationen

Basisimplementierung: `CommandHandlerBase<T>` orchestriert `Validation` → `Execution` → `Status-Persistierung`.

8.3 Fehlerbehandlung

8.3.1 Exception-Hierarchie

Alle Framework-Exceptions erben von `EGovException` und tragen einen strukturierten code (String) sowie optionale `additionalInfo` (Map).

`EGovException`

```
├─ NoPermissionForEntityException → HTTP 403
├─ ActionNotAllowedException     → HTTP 405
├─ UnauthorizedException         → HTTP 401
└─ SystemException              → HTTP 500
```

`RuntimeException`

```
└─ ValidationException           (intern, kein HTTP-Mapping)
```

8.3.2 Einheitliche Fehlerantwort

`GenericExceptionHandler` (`@ControllerAdvice`) fängt alle Exceptions und wandelt sie in ein einheitliches `ResponseExceptionBody-DTO` um:

```
{
  "message": "No permission for entity 'Person' (operation: READ)",
  "exceptionId": "f3a1bc42-...",
  "errorCode": "EGOV_FRM_0012",
  "additionalInfo": { "tableName": "person", "userId": "u42" },
  "stackTrace": null
}
```

Stacktrace wird nur ausgegeben wenn `egov.exception-log.debug=true` gesetzt ist.

8.3.3 Fehler-Code-Schema

Code	Bedeutung
EGOV_FRM_0000	Generischer Framework-Fehler
EGOV_FRM_0001	Ungültige Daten
EGOV_FRM_0012	Keine Berechtigung auf Entity
EGOV_FRM_0013	Kein Executor gesetzt
EGOV_FRM_0014	Ungültige Action
EGOV_FRM_0015	ZooKeeper-Fehler
EGOV_PRS_0000	ID-Limit überschritten (LUID)

8.4 Persistenz

8.4.1 Repository-Pattern

Alle Domänen-Entitäten der consuming App erweitern `BaseRepositoryImpl<T>`. Diese Basisklasse stellt generische CRUD-Operationen bereit und delegiert den Query-Bau an den `QueryBuilder`.

8.4.2 Filter-DSL

REST-Anfragen können einen Filterausdruck als String übergeben. Der `FilterParser` und `Tokenizer` übersetzen diesen String in einen `Abstract Syntax Tree`; der `QueryBuilder` erzeugt daraus native SQL ohne SQL-Injection-Risiko.

8.4.3 Multi-Datenbankunterstützung

Property	Werte	Standardwert
<code>egov.persistence.database.type</code>	POSTGRESQL, SQLSERVER	— (Pflichtfeld)
<code>egov.persistence.database.naming.style</code>	CAMEL_CASE, SNAKE_CASE	CAMEL_CASE

`DatabaseConfigurationService` liest diese Werte und konfiguriert Dialekt und Naming- Strategie beim Start.

8.4.4 Datenbankmigrationen

Das Framework liefert eigene Flyway-Skripte für seine IAM- und Command-Tabellen mit. Consuming Apps fügen applikationsspezifische Skripte hinzu. Die Flyway-Schema-Filterung erfolgt über `EgovSchemaFilterProvider`, damit Framework-Migrations nicht mit App- Migrations kollidieren.

8.5 Konfiguration

Alle Framework-Properties sind unter dem Namespace `egov.*` in der `application.yml` der consuming App gruppiert. Das Framework liefert `egov-default.yml` mit sicheren Defaults mit.

Zentrale Properties:

Property	Default	Beschreibung
<code>egov.persistence.database.type</code>	—	Datenbanktyp (POSTGRESQL / SQLSERVER)
<code>egov.persistence.database.naming.style</code>	CAMEL_CASE	Spaltenbenennungsschema
<code>egov.persistence.luid.max-ids-per-tick</code>	10000	IDs pro Sekunde pro Segment
<code>egov.persistence.luid.max-segments</code>	100	Anzahl paralleler Pods/JVMs
<code>egov.persistence.luid.start-year</code>	2025	Basisjahr für LUID-Berechnung
<code>egov.admin.is.activated</code>	false	Admin-Modul aktivieren
<code>egov.permission.log.activated</code>	false	Permission-Entscheidungen loggen
<code>egov.exception-log.debug</code>	false	Stacktrace in Fehlerantworten
<code>egov.service.discovery.activated</code>	false	ZooKeeper-Service-Discovery

Property	Default	Beschreibung
egov.service.discovery.zookeeper.hostname	localhost	ZooKeeper-Hostname
egov.service.discovery.zookeeper.port	2181	ZooKeeper-Port
egov.subscriber.core-pool-size	5	ThreadPool für Event-Subscriptions
egov.subscriber.max-pool-size	20	Max-Threads für Event-Subscriptions

8.6 Logging

Das Framework verwendet SLF4j mit Logback als Implementierung. Alle relevanten Klassen tragen @Slf4j (Lombok). Kein MDC-Einsatz im Framework-Kern — die consuming App ist für das Anreichern von Logs (z.B. mit userId oder requestId) verantwortlich.

Log-Level-Konfiguration (Auszug):

```
logging:
  level:
    ch.ti8m.egov.framework.exceptionhandling.exceptionhandlers: DEBUG
    ch.ti8m.egov.framework.validation.engine.FieldValueExtractorComponent: DEBUG
```

Klassen mit relevantem Logging: ContextInterceptor, PermissionBuilder, GenericExceptionHandler, CommandHandlerBase, LUIDGenerator, ZookeeperConnector.

ContextInterceptor.preHandle() extrahiert neben der userId auch Request-Parameter (filter, sort, offset, limit, page, size, search) in den DataHolder. afterCompletion() bereinigt den ThreadLocal.

8.7 Internationalisierung

Das optionale e-gov-mdm-Modul stellt mehrsprachige Stammdaten (Vocabularies) bereit.

8.7.1 Datenstruktur

```
Vocabulary
├─ MasterDataGenericEntity (pro Eintrag und Sprache)
│   ├── code (fachlicher Schlüssel)
│   ├── languageCode (de, fr, it, en, ...)
│   ├── shortName
│   ├── longName
│   ├── validFrom
│   └─ validTo
```

LanguageDefinition verknüpft ein Vocabulary mit den verfügbaren Sprachcodes.

8.7.2 Sprachauswahl

Die aktive Sprache wird in folgender Reihenfolge ermittelt:

1. Content-Language-Header des HTTP-Requests (gesetzt durch ContextInterceptor)
2. `UserLanguageProvider.getLanguage()` — Default-Implementierung: `Locale.getDefault()`

Consuming Apps können `UserLanguageProvider` als Spring-Bean überschreiben, um die Sprache aus dem Nutzerprofil oder einem anderen Request-Attribut zu lesen.

API-Ausgabe: `ResponseTranslation` mit `shortName` und `longName` pro Sprache.

8.8 ID-Generierung (LUID)

Das optionale `e-gov-luid`-Modul stellt eine clustersichere, sortierbare ID-Strategie bereit, die keine Datenbanksequenz benötigt.

8.8.1 Algorithmus (LUIDGenerator)

$$ID = tickStep + incFromBase$$

$$tickStep = (\text{Millisekunden seit startYear}) / 1000 * \text{maxIDsPerTick} * \text{maxSegments}$$

$$incFromBase = \text{maxIDsPerTick} * \text{segmentId} + [0 \dots \text{maxIDsPerTick} - 1]$$

- **segmentId:** eindeutiger Index des Pods/der JVM-Instanz (0–99 bei Default)
- **maxIDsPerTick:** Anzahl IDs, die pro Sekunde und Segment vergeben werden können
- **maxSegments:** maximale Anzahl paralleler Instanzen im Cluster

`LUIDGenerator` ist als Hibernate `IdentifizierGenerator` implementiert, `generate()` ist synchronized. Konfigurierbar über `egov.persistence.luid.*-Properties` (siehe Konfiguration).

Eigenschaften: zeitbasiert, sortierbar, gap-tolerant, kein Single-Point-of-Failure, kein Round-Trip zur Datenbank.

8.9 Testbarkeit

8.9.1 Test-Pyramide

Ebene	Technologie	Beispiele
Unit-Tests	JUnit 5	<code>BaseRepositoryImplTest</code> , <code>CommandHandlerBaseTest</code> , <code>FilterBuilderTest</code>
Integrationstests	JUnit 5 + Testcontainers	<code>BaseRepositoryImplIT</code> , <code>SubscriptionServiceIT</code>
ZooKeeper-Tests	Testcontainers (zookeeper:3.9.3)	<code>ZookeeperTestContainer-Basisklasse</code>

8.9.2 Testcontainers-Konfiguration

Datenbank-Integrationstests verwenden das JDBC-URL-Schema von Testcontainers:

```
# application-test.yml
spring:
  datasource:
    url: jdbc:tc:postgresql:17.4:///integration-tests-db
    driver-class-name: org.testcontainers.jdbc.ContainerDatabaseDriver
  jpa:
    hibernate:
      ddl-auto: create-drop
```

Weitere Profile: application-h2.yml (In-Memory), application-test-sqlserver.yml, application-test-zookeeper.yml, application-test-logging-debug.yml.

9 Architekturentscheidungen

Architecture Decision Records (ADRs) dokumentieren die wichtigen Entscheidungen, die die Architektur des e-gov-framework prägen. Jeder ADR hält Kontext, getroffene Entscheidung, betrachtete Alternativen und Konsequenzen fest.

9.1 ADR-001: Framework als eingebettete Java-Bibliothek, nicht als eigenständiger Service

Status: Akzeptiert

Datum: 2023 (Projektgründung)

9.1.1 Kontext

E-Government-Fachapplikationen verschiedener Schweizer Behörden teilen denselben Bedarf an Querschnittsfunktionen: row-based Security, Audit-Trail, Stammdaten, Persistence-Abstraktion. Die Frage war, wie diese Funktionen bereitgestellt werden sollen.

Alternativen:

Kriterium	Bibliothek (JAR)	Sidecar / Microservice	SaaS-Plattform
Deploymentkomplexität	Gering (eine JVM)	Hoch (zusätzlicher Prozess)	Abhängig von Anbieter
Latenz	Keine (in-process)	Netzwerk-Roundtrip (~ms)	Netzwerk-Roundtrip
Permission-Check			
Hoheit über Infrastruktur	Vollständig bei Behörde	Teilweise	Keine
Compliance / Data Residency	Einfach (Daten verlassen nie die JVM)	Möglich	Kritisch
Versionierung je Applikation	Unabhängig via BOM	Gemeinsamer Release-Zyklus	Vorgeschrieben

9.1.2 Entscheidung

Das Framework wird als Maven-Bibliothek distribuiert. Jede consuming App zieht das Framework über das BOM als Abhängigkeit und betreibt es in derselben JVM.

9.1.3 Konsequenzen

Positiv: - Kein Netzwerk-Overhead bei Permission-Checks (kritisch für Row-based Security) - Jede Behörde kann unabhängig upgraden oder pinnen - Keine zusätzliche Laufzeitinfrastuktur notwendig - Vollständige Datenisolation pro Behörde garantiert

Negativ: - Framework-Logik läuft in derselben JVM wie Applikationslogik — Fehler im Framework können die gesamte Applikation betreffen - Consuming Apps tragen die Last des Upgrades (pom.xml-Änderung, Migrationstests) - Kein zentrales Monitoring über alle Deployments hinweg möglich

9.2 ADR-002: Native JDBC für die Query-Schicht statt JPA Criteria API

Status: Akzeptiert

Datum: 2023

9.2.1 Kontext

Das Framework muss jede SELECT-Abfrage automatisch um rollenbasierte WHERE-Klauseln erweitern (Row-based Security) und eine eigene Filter-DSL aus REST-Parametern umsetzen. JPA/Hibernate übernimmt das Entity-Mapping. Für die Query-Schicht standen zwei Ansätze zur Auswahl.

Alternativen:

Kriterium	JPA Criteria API	Native JDBC / SQL
Kontrolle über WHERE-Klauseln	Eingeschränkt (Hibernate-Abstraktion)	Vollständig
Lesbarkeit komplexer Queries	Schlecht (Boilerplate)	Gut (klares SQL)
Multi-DB-Unterstützung	Via Hibernate-Dialekte	Manuell je Dialekt
Performance	Overhead durch Metadaten-Lookups	Minimal
Permission-Filter injizieren	Schwierig (kein direkter SQL-Zugriff)	Trivial (String-Konkatenation)

9.2.2 Entscheidung

Die `BaseRepositoryImpl` und der `QueryBuilder` verwenden natives JDBC für alle Leseabfragen. JPA/Hibernate bleibt für Entity-Mapping und Schreiboperationen (`saveWithTx`) zuständig.

9.2.3 Konsequenzen

Positiv: - `PermissionBuilder` kann WHERE-Klauseln direkt als SQL-Fragment injizieren - Filter-DSL lässt sich sauber auf SQL abbilden ohne Criteria-API-Umweg - Queries sind lesbar und debuggbar - Performance-Ziel (<500 ms für 10 000 Entitäten) erreichbar

Negativ: - Zwei Persistenzmechanismen (JPA + JDBC) müssen koordiniert werden - SQL-Dialekte (PostgreSQL vs. SQL Server) müssen in `DatabaseConfigurationService` manuell behandelt werden - `BaseRepositoryImpl` ist deswegen komplex

9.3 ADR-003: Command-Pattern für alle Schreiboperationen

Status: Akzeptiert

Datum: 2023

9.3.1 Kontext

Behörden-Applikationen unterliegen strikten Audit-Anforderungen: Jede fachliche Aktion muss nachvollziehbar sein (wer hat was wann getan). Ausserdem müssen Schreiboperationen validierbar und rollback-fähig sein. Es wurden zwei Modelle verglichen.

Alternativen:

Kriterium	Command-Pattern (persistiert)	Direkte Service-Calls
Audit-Trail	Vollständig (jede Aktion als Datenbankzeile)	Nur via separates Log
Validierung vor/nach Ausführung	Steuerbar via ExecutionPlan	Inline, nicht standardisiert
Nachvollziehbarkeit für Behörde	Hoch (egov_cmd_command-Tabelle abfragbar)	Gering
Komplexität für consuming Apps	Mittel (BaseExecutor, DomainCommandFactory)	Gering
Distributed Execution	Möglich (RemoteCommandExecutor)	Nicht vorgesehen

9.3.2 Entscheidung

Jede fachliche Schreiboperation wird als Command-JPA-Entity angelegt und mit einem Status-Lifecycle (INITIALIZED → VALIDATED → EXECUTED oder HARD_VALIDATION_FAILED) in der Tabelle egov_cmd_command persistiert. BaseExecutor-Implementierungen orchestrieren den Ablauf.

9.3.3 Konsequenzen

Positiv: - Lückenloser Audit-Trail ohne Zusatzkonfiguration für consuming Apps - Validierungsregeln (RuleSet) werden deklarativ via Annotationen definiert - ExecutionPlan ermöglicht flexible Validierungsstrategien pro Use-Case - Erweiterbar auf verteilte Ausführung über mehrere Instanzen (ZooKeeper)

Negativ: - Jede Schreiboperation erzeugt einen zusätzlichen Datenbank-Insert (Command-Tabelle) - Consuming Apps müssen DomainCommandFactory und BaseExecutor ableiten und kennen - Es besteht so das Risiko, dass das Framework falsch verwendet wird.

9.4 ADR-004: Row-based Security als opt-out statt opt-in

Status: Akzeptiert

Datum: 2023

9.4.1 Kontext

Datenpannen in Behörden entstehen oft durch vergessene oder inkorrekte Berechtigungsprüfungen in Applikationscode. Das Framework sollte sicherstellen, dass kein Entwickler einer consuming App versehentlich ungeschützte Abfragen absetzt.

Alternativen:

Ansatz	Beschreibung	Risiko
Opt-in (explizit)	Entwickler ruft <code>PermissionBuilder</code> manuell auf	Vergessen möglich → Datenpanne
Opt-out (Default)	<code>QueryBuilder</code> fügt immer Permission-Filter ein; explizites Abschalten nötig	Kein versehentliches Bypass
Applikationsschicht	Permission-Prüfung liegt komplett in der consuming App	Fehleranfällig, kein Framework-Beitrag

9.4.2 Entscheidung

Der `QueryBuilder` bindet den `PermissionBuilder` immer automatisch ein. Jede `BaseRepository-Impl.find()`-Abfrage ist per Default row-level-gefiltert. Ein expliziter Bypass ist nur für interne/Admin-Operationen über den Admin-Modus (`egov.admin.is.activated`) vorgesehen.

9.4.3 Konsequenzen

Positiv: - Ein Sachbearbeiter sieht niemals Datensätze ohne Berechtigung — auch nicht durch Programmierfehler in der consuming App - Sicherheitsverantwortliche können die Filterlogik zentral in `egov_iam_permission_definition` administrieren ohne App-Deployments

Negativ: - Höherer Aufwand beim initialen Setup: Permission-Definitionen müssen vollständig in der IAM-Tabelle gepflegt sein, bevor Daten sichtbar sind - Interne Batch-Jobs oder Admin-Operationen müssen den Admin-Modus explizit aktivieren

9.5 ADR-005: Vorgelagertes IAM — kein eigener Authentifizierungsstack

Status: Akzeptiert

Datum: 2023

9.5.1 Kontext

Schweizer Behörden betreiben eigene Identity-Management-Systeme (Azure AD, LDAP, eigene IdPs). Ein Framework, das selbst Authentifizierung implementiert, würde entweder diese Systeme duplizieren oder Integrationsaufwand in jeder Applikation erzwingen.

Alternativen:

Ansatz	Beschreibung
Eigener JWT-Stack	Framework validiert Tokens selbst, kennt Claims-Schema
Spring Security Integration	Framework konfiguriert Spring Security mit konfigurierbarem IdP

Ansatz	Beschreibung
Vorgelagertes IAM (gewählt)	F5-Firewall setzt EGOV_TOKEN; Framework vertraut dem Cookie

9.5.2 Entscheidung

Das Framework führt keine Authentifizierung durch. Es vertraut darauf, dass der vorgelagerte Stack (Azure AD → F5 → EGOV_TOKEN-Cookie) eine authentifizierte userId liefert. Das UserIdProvider-Interface ist der einzige Integrationspunkt: Consuming Apps implementieren es einmalig für ihren spezifischen Cookie/Header.

9.5.3 Konsequenzen

Positiv: - Framework ist unabhängig vom konkreten Identity-Provider der Behörde - Keine doppelte Token-Validierungslogik in Framework und vorgelagertem Stack - Consuming Apps können UserIdProvider auf beliebige Auth-Quellen (JWT-Claim, Header, Session) mappen

Negativ: - Framework kann keine Claims-basierten Attribute (Abteilung, Berechtigungsgruppe) direkt aus dem Token lesen — consuming App muss das ggf. in UserIdProvider auflösen - Framework vertraut blind dem Cookie-Inhalt; korrekte TLS-Terminierung und Cookie-Sicherheit liegen vollständig beim vorgelagerten Stack

9.6 ADR-006: Unterstützung von PostgreSQL und SQL Server parallel

Status: Akzeptiert

Datum: 2023

9.6.1 Kontext

Verschiedene Behörden haben unterschiedliche Infrastrukturvorgaben: Einige betreiben PostgreSQL-Cluster (bevorzugt, Open Source, günstiger), andere sind durch Rahmenverträge oder IT-Governance an Microsoft SQL Server gebunden. Ein Framework, das nur eine Datenbank unterstützt, würde Behörden mit SQL-Server-Vorgaben ausschliessen.

Alternativen:

Kriterium	Nur PostgreSQL	Nur SQL Server	Dual-DB (gewählt)	Polyglot (+ H2 / MySQL)
Abdeckung Behördenanforderungen	Unvollständig (MSSQL-Kunden ausgeschlossen)	Unvollständig (PG-Kunden ausgeschlossen)	Vollständig (alle aktuellen Kunden)	Überabdeckung, unnötige Komplexität
Implementierungsaufwand	Gering	Gering	Mittel (2 Dialekte)	Hoch (n Dialekte)
CI-Aufwand (Testcontainers)	1 DB-Typ	1 DB-Typ	2 DB-Typen	n DB-Typen
Wartbarkeit Dialekt-Code	Hoch	Hoch	Mittel	Gering

9.6.2 Entscheidung

Das Framework unterstützt beide Datenbanktypen vollständig. DatabaseConfigurationService liest `egov.persistence.database.type` und konfiguriert: - Hibernate-Dialekt - Spalten-Namensschema (CAMEL_CASE / SNAKE_CASE) - Datenbank-spezifische SQL-Fragmente im QueryBuilder - Flyway-Skripte pro Datenbank-Typ

9.6.3 Konsequenzen

Positiv: - Framework ist für alle Behörden einsetzbar, unabhängig von ihrer DB-Infrastruktur - Wechsel von PostgreSQL zu SQL Server (oder umgekehrt) erfordert nur Property-Änderung

Negativ: - Jede Änderung am Query-Layer muss gegen beide Datenbanken getestet werden - QueryBuilder und Flyway-Skripte müssen Dialekt-Unterschiede kennen und behandeln - Erhöhter CI-Aufwand: Integrationstests laufen gegen beide Datenbanken via Testcontainers

9.7 ADR-007: Spring AI + Azure OpenAI für LLM-gestützte Regelgenerierung

Status: Akzeptiert (experimentell)

Datum: 2024

9.7.1 Kontext

Die Pflege von Validierungsregeln in `egov_iam_rule_set` ist aufwändig und erfordert SQL-/Framework-Kenntnisse. Die Idee: Admins beschreiben Regeln in natürlicher Sprache, das System generiert die technische Regeldarstellung automatisch.

Für die LLM-Integration standen zwei Optionen:

Kriterium	Spring AI (Abstraktionsschicht)	Direktes OpenAI SDK
LLM-Austauschbarkeit	Hoch (Providerwechsel via Config)	Gering (Vendor-Lock-in)
Reife Integration in Spring Boot	Milestone-Release (1.0.0-M6) Native	Stabil (GA) Manuell
Zukünftige On-Premise-Option	Möglich (lokales LLM via Spring AI)	Schwierig

9.7.2 Entscheidung

Das Modul `e-gov-ai-rule-generation` verwendet Spring AI als Abstraktionsschicht und Azure OpenAI (gpt-4o) als Backend. Der Prompt enthält die Grammatik der erlaubten Validierungsfunktionen; Rule-Translator konvertiert die LLM-Antwort in das Framework-Regelformat.

Das Modul wird bis zum GA-Release von Spring AI als **experimentell** gekennzeichnet.

9.7.3 Konsequenzen

Positiv: - Admins können Regeln aus Freitext generieren ohne SQL-Kenntnisse - Spring AI ermöglicht späteren Wechsel auf On-Premise-LLM ohne Code-Änderung

Negativ: - Breaking Changes zwischen Milestones möglich (→ Risiko R-4) - Azure OpenAI-Abhängigkeit erfordert Netzwerkzugang und API-Key-Verwaltung - Modul ist bisher ohne Testabdeckung (→ technische Schuld TD-1) - Azure OpenAI erfordert Cloud-Egress — dies steht in Konflikt mit der Randbedingung «Kubernetes On-Premise» (→ Sektion 2); der Konflikt ist bewusst akzeptiert: e-gov-ai-rule-generation ist optional, Behörden ohne Cloud-Freigabe lassen das Modul weg

9.8 ADR-008: LUID als cluster-sichere ID-Generierungsstrategie

Status: Akzeptiert

Datum: 2024

9.8.1 Kontext

Fachapplikationen, die auf Kubernetes mit mehreren Replikas laufen, brauchen eine ID-Generierungsstrategie, die ohne zentrale Koordination skaliert. Standard-Alternativen wurden bewertet:

Strategie	Sortierbarkeit	DB-Abhängigkeit	Cluster-sicher	Lücken
UUID v4	Nicht sortierbar	Keine	Ja	Keine
DB-Sequenz	Sortierbar	Ja (Single Point)	Nur mit DB-Failover	Möglich
ULID	Sortierbar (ms-Präzision)	Keine	Bedingt (Kollision möglich)	Keine
LUID (gewählt)	Sortierbar (sek-Präzision)	Keine	Ja (via segmentId)	Toleriert

9.8.2 Entscheidung

Das Modul e-gov-luid stellt LUIDGenerator bereit — einen Hibernate IdentifierGenerator, der zeitbasierte, sortierbare IDs ohne Datenbankzugriff generiert. Jeder Pod erhält eine eindeutige segmentId (0–99), die Kollisionen zwischen Instanzen ausschliesst.

```
ID = ((Sekunden seit 2025-01-01) * maxIDsPerTick * maxSegments)
      + (maxIDsPerTick * segmentId)
      + laufende Nummer [0 ... maxIDsPerTick - 1]
```

9.8.3 Konsequenzen

Positiv: - Keine Datenbankabfrage bei ID-Generierung → Durchsatz und Latenz verbessert - IDs sind sortierbar → natürliche Paginierung und Index-Effizienz - Skaliert bis 100 parallele Pods bei Defaultkonfiguration

Negativ: - segmentId muss beim Deployment korrekt konfiguriert sein — falscher Wert kann Kollisionen verursachen - IDs sind nicht zufällig (Information Disclosure: Erstellungszeitpunkt ableitbar) - Fehlkonfiguration (zwei Pods mit gleicher segmentId) führt zu Kollisionen ohne sofortige Fehlermeldung

10 Qualitätsanforderungen

Die drei wichtigsten Qualitätsziele sind in [Sektion 1.2](#) beschrieben. Dieses Kapitel enthält den vollständigen Qualitätsbaum und messbare Szenarien für alle relevanten Qualitätskategorien.

10.1 Qualitätsbaum

Kategorie	Qualitätsanforderung	Priorität	Szenario
Wartbarkeit	Neue CRUD-Domäne in einem Arbeitstag integrierbar	Hoch	QS-01
Wartbarkeit	Breaking Changes erkennbar vor dem Upgrade	Hoch	QS-02
Performance	Gefilterte, paginierte Liste von 10 000 Entitäten in < 500 ms	Hoch	QS-03
Sicherheit	Kein Benutzer sieht unberechtigt Datensätze	Hoch	QS-04
Sicherheit	Jede Schreiboperation ist nachvollziehbar	Hoch	QS-05
Zuverlässigkeit	Pod-Ausfall ohne Datenverlust und ohne manuelle Intervention	Mittel	QS-06
Zuverlässigkeit	Framework-Startfehler werden frühzeitig erkannt	Mittel	QS-07
Portabilität	Datenbankwechsel ohne Code-Änderung in consuming Apps	Mittel	QS-08
Betreibbarkeit	Fehlkonfiguration wird beim Start gemeldet	Mittel	QS-09
Testbarkeit	Integrationstests ohne produktive Infrastruktur ausführbar	Niedrig	QS-10
Stammdatenkonsistenz	Alle Fachapplikationen verwenden identische, mehrsprachige Codelisten	Mittel	QS-11

10.2 Qualitätsszenarien

10.2.1 QS-01: Neue CRUD-Domäne in kurzer Zeit

Aspekt	Beschreibung
Kategorie	Wartbarkeit / Developer Experience
Stimulus	Ein Entwickler einer Behörde erhält den Auftrag, eine neue Domäne (Entity, Repository, Command, REST-Endpoint) zu implementieren
Umgebung	Consuming App mit konfiguriertem e-gov-framework; Entwickler kennt die Dokumentation und die Demo-Applikation
Artefakt	Framework-Kern (BaseRepositoryImpl, BaseExecutor, DomainCommandFactory)
Reaktion	Die Domäne ist vollständig integriert: persistierbar, permission-gefiltert, audit-getrailed, per REST erreichbar
Metrik	Aufwand ≤ 1 Arbeitstag (8 h) für einen Entwickler mit Java/Spring-Erfahrung

Architektonische Massnahmen: BOM für konsistentes Dependency-Management; BaseRepository-Impl/BaseExecutor-Basisklassen; Convention over Configuration via Spring Boot Auto-Konfiguration; egov-framework-demo als Referenzimplementierung.

10.2.2 QS-02: Breaking Changes vor dem Upgrade erkennbar

Aspekt	Beschreibung
Kategorie	Wartbarkeit / Evolvierbarkeit
Stimulus	Das Framework-Team veröffentlicht eine neue Version
Umgebung	Consuming App; Entwickler prüft Kompatibilität vor dem BOM-Upgrade
Artefakt	Maven BOM, CHANGELOG, Semantic Versioning
Reaktion	Breaking Changes sind als Major-Version-Bump (SemVer) und im CHANGELOG dokumentiert; Minor/Patch-Upgrades sind ohne Anpassung in der consuming App möglich
Metrik	100 % der Breaking Changes werden als Major-Version veröffentlicht; CHANGELOG enthält Migration-Hinweise

Architektonische Massnahmen: Conventional Commits steuern automatische Versionsvergabe via Semantic Release; CHANGELOG wird automatisch generiert; Artifactory als versioniertes Repository.

10.2.3 QS-03: Gefilterte Listenabfrage unter 500 ms

Aspekt	Beschreibung
Kategorie	Performance / Effizienz
Stimulus	Ein Sachbearbeiter lädt eine gefilterte, paginierte Liste
Umgebung	Produktivsystem unter Normallast; Tabelle enthält 10 000 Entitäten; Benutzer hat eingeschränkte Rollenberechtigungen
Artefakt	QueryBuilder, PermissionBuilder, FilterParser, Datenbankindex
Reaktion	Die gefilterte, permission-bereinigte, paginierte Ergebnisliste wird korrekt zurückgeliefert
Metrik	Gesamtantwortzeit (inkl. Permission-Filter) ≤ 500 ms (p95)

Architektonische Massnahmen: Nativer JDBC statt JPA Criteria API (kein ORM-Overhead); Permission-Filter direkt in SQL (kein Post-Filter in Java); Indizes auf egov_iam_role_membership (userId, valid_from, valid_to).

Bekannte Grenze: Tokenizer.java wirft einen Stack Overflow bei IN-Conditions mit ca. 1 300+ Werten (→ Risiko R-1).

Aspekt	Beschreibung
--------	--------------

10.2.4 QS-04: Kein Benutzer sieht unberechtigt Datensätze

Aspekt	Beschreibung
Kategorie	Sicherheit / Vertraulichkeit
Stimulus	Ein Sachbearbeiter sendet eine Listeanfrage für Entitäten, für die er keine Berechtigung hat
Umgebung	Produkktivsystem; Benutzer ist authentifiziert, aber nicht für alle Datensätze autorisiert
Artefakt	PermissionBuilder, BaseRepositoryImpl, egov_iam_permission_definition
Reaktion	Die Antwort enthält ausschliesslich Datensätze, für die eine permission_definition mit der Rolle des Benutzers existiert; unberechtigte Datensätze werden weder zurückgegeben noch als fehlend signalisiert
Metrik	0 unberechtigte Datensätze in jeder Antwort; nachweisbar durch Integrationstests mit verschiedenen Rollen

Architektonische Massnahmen: Row-based Security als opt-out (ADR-004); PermissionBuilder ist ohne expliziten Admin-Modus nicht umgehbar.

10.2.5 QS-05: Jede Operation nachvollziehbar

Aspekt	Beschreibung
Kategorie	Sicherheit / Nachvollziehbarkeit / Compliance
Stimulus	Ein Sicherheitsverantwortlicher oder Prüfer möchte nachvollziehen, wer wann welche Datenänderung vorgenommen hat
Umgebung	Produkktivsystem; die Fachapplikation nutzt BaseExecutor für alle Schreiboperationen
Artefakt	Command-Tabelle (egov_cmd_command), DomainCommandFactory
Reaktion	Jede Schreiboperation ist als Zeile in egov_cmd_command protokolliert: Benutzer, Zeitstempel, Aktion, Status (EXECUTED / HARD_VALIDATION_FAILED)
Metrik	100 % aller Schreiboperationen, die über BaseExecutor laufen, sind in der Command-Tabelle eingetragen

Architektonische Massnahmen: Command-Pattern für alle Schreiboperationen (ADR-003); INITIALIZED-Status wird vor der Ausführung persistiert, sodass auch abgebrochene Operationen sichtbar sind.

Bekannte Grenze: NonTxExecutor-Operationen laufen ausserhalb von Spring-Transaktionen; bei einem Pod-Ausfall während solcher Operationen ist die Rollback-Garantie nicht gewährleistet (→ Risiko R-2). Dies wird in Framework 2.0 adressiert.

Aspekt	Beschreibung
--------	--------------

10.2.6 QS-06: Pod-Ausfall ohne Datenverlust

Aspekt	Beschreibung
Kategorie	Zuverlässigkeit / Fehlertoleranz
Stimulus	Ein Kubernetes-Pod, der das Framework ausführt, fällt während einer Schreiboperation aus
Umgebung	Produkktivsystem; Kubernetes-Cluster mit mehreren Replikas; PostgreSQL mit synchronem Commit
Artefakt	Command-Tabelle, Spring-Transaktionsmanagement, Kubernetes Liveness/Readiness Probes
Reaktion	Neue Anfragen werden durch Kubernetes auf verbleibende Pods umgeleitet; halbfertige Transaktionen werden durch die Datenbank zurückgerollt; der Command-Eintrag verbleibt im Status INITIALIZED und ist damit als unvollständig erkennbar
Metrik	Kein stiller Datenverlust; Wiederherstellungszeit < 30 s (Kubernetes-Probe-Intervall)

Architektonische Massnahmen: Command-Status INITIALIZED wird vor der Ausführung persistiert; Spring-Transaktionen stellen atomares Rollback sicher; LUID-IDs sind ohne DB-Koordination erzeugbar.

Bekannte Grenze: NonTxExecutor-Operationen laufen ausserhalb von Spring-Transaktionen; ein Pod-Ausfall während solcher Operationen kann zu inkonsistenten Datenbankzuständen führen (→ Risiko R-2). Dies wird in Framework 2.0 adressiert.

10.2.7 QS-07: Datenbankwechsel ohne Code-Änderung

Aspekt	Beschreibung
Kategorie	Portabilität / Flexibilität
Stimulus	Eine Behörde wechselt ihre Infrastruktur von PostgreSQL zu SQL Server (oder umgekehrt)
Umgebung	Bestehende consuming App; neues Datenbankschema wird mit Flyway migriert
Artefakt	DatabaseConfigurationService, QueryBuilder, Flyway-Skripte
Reaktion	Die consuming App läuft nach Anpassung von <code>egov.persistence.database.type</code> ohne Änderung an Java-Code oder Queries korrekt gegen die neue Datenbank
Metrik	0 Code-Änderungen in der consuming App beim DB-Typwechsel; alle Integrationstests bestehen gegen beide Datenbanktypen

Architektonische Massnahmen: Dual-DB-Support (ADR-006); DatabaseConfigurationService kapselt alle Dialekt-Unterschiede.

10.2.8 QS-08: Integrationstests ohne produktive Infrastruktur

Aspekt	Beschreibung
Kategorie	Testbarkeit
Stimulus	Ein Entwickler führt die Integrationstests lokal oder in der CI-Pipeline aus
Umgebung	Entwickler-Workstation oder GitLab-CI-Runner; kein Zugriff auf produktive Datenbanken
Artefakt	Testcontainers-Konfiguration, <code>application-test.yml</code>
Reaktion	Die Integrationstests starten PostgreSQL und ggf. ZooKeeper als kurzlebige Docker-Container und laufen vollständig durch
Metrik	Alle Integrationstests laufen ohne externe Infrastruktur; Ausführungszeit in CI < 10 Minuten

Architektonische Massnahmen: Testcontainers mit JDBC-URL-Protokoll (`jdbc:tc:postgresql:...`); separate Test-Profile für PostgreSQL, SQL Server, H2 und ZooKeeper.

10.2.9 QS-09: Stammdatenkonsistenz über Applikationsgrenzen

Aspekt	Beschreibung
Kategorie	Stammdatenkonsistenz / Interoperabilität
Stimulus	Fachapplikationen können auf denselben Codelisten-Typ zugreifen (z.B. Nationalität)
Umgebung	Produktivsystem; beide Apps nutzen das e-gov-mdm-Modul mit gemeinsamer Datenbank
Artefakt	e-gov-mdm, e-gov-mdm-api, egov-mdm-vocabulary-Tabellen
Reaktion	Beide Applikationen liefern identische, mehrsprachige Codelisten-Werte für denselben Code-Typ
Metrik	0 Inkonsistenzen zwischen Applikationen bei Codelisten-Abfragen; nachweisbar durch cross-App-Integrationstests

Architektonische Massnahmen: Zentrales MDM-Modul als Single Source of Truth; e-gov-mdm-api stellt REST-Endpunkte bereit; Flyway-Migrations-Skripte sichern konsistente Codelisten-Initialisierung.

10.3 Mengengerüst

Kennzahl	Wert	Bemerkung
Maximale Entitäten pro Tabelle (Richtwert)	10 000	
Maximale IN-Filter-Werte	ca. 1 300	Technisches Limit Tokenizer (Risiko R-1)
Maximale parallele Pod-Instanzen	100	LUID-Konfiguration <code>maxSegments</code>
Unterstützte Datenbanktypen	2 (PostgreSQL, SQL Server)	
Unterstützte Sprachen (MDM)	de, fr, it, en	Konfigurierbar via <code>UserLanguageProvider</code>

11 Risiken und technische Schulden

Bekannte technische Risiken und Schulden — geordnet nach Eintrittswahrscheinlichkeit und Auswirkung.

11.1 Risiken

ID	Risiko	Wahrscheinlichkeit	Auswirkung	Massnahme
R-1	Performance-Engpass: Stack Overflow bei grossen IN-Filter-Ausdrücken — Tokenizer .java wirft bei IN-Conditions mit ca. 1300+ Werten einen Stack Overflow	mittel	hoch	Pagination oder serverseitige Limit-Prüfung vor der Query-Verarbeitung einführen; bekannte Schwelle dokumentieren
R-2	Transaktionsverwaltung ausserhalb Spring Boot — Teile des Frameworks verwalten Transaktionen eigenständig, ohne Spring-Transaktionsmechanismen zu nutzen; Verhalten bei Rollback, Nested Transactions und AOP-Proxying ist fehleranfällig; betrifft Qualitätsziele QS-05 (Audit-Trail) und QS-06 (Pod-Failover)		hoch	Mit Framework 2.0 vollständige Spring-Transaktionsintegration einführen (in Umsetzung)

ID	Risiko	Wahrscheinlichkeit	Auswirkung	Massnahme
R-3	Key-Person-Dependency — Kernwissen über Framework-Internals (Persistenz-Abstraktion, ZooKeeper-Integration, AI-Modul) konzentriert sich	mittel	hoch	Wissenstransfer und Pair-Programming forcieren; kritische Designentscheidungen in ADRs (Sektion 9) dokumentieren
R-4	Spring AI Milestone-Release — Dependency spring-ai 1.0.0-M6 ist kein stabiles Release; API-Breaking-Changes zwischen Milestones möglich	mittel	mittel	

12 Glossar

Wichtige Fach- und Technikbegriffe alphabetisch sortiert. Domänenspezifische Framework-Konzepte, Abkürzungen und externe Systembezeichnungen.

12.1 Framework-Konzepte

Begriff	Definition
ArchivedModifiableEntity	JPA-Basisklasse für Soft-Delete; Entitäten werden archiviert statt physisch gelöscht. Alle Abfragen filtern archivierte Einträge automatisch aus.
BaseExecutor	Abstraktes Interface, das eine <code>execute(Command)</code> -Methode vorschreibt. Koordiniert den Command-Lifecycle: Validation → Ausführung → Status-Persistierung. Zwei Varianten: <code>Executor</code> (transaktional) und <code>NonTxExecutor</code> (nicht-transaktional).
BaseRepositoryImpl	Abstrakte Basisklasse für alle Domänen-Repositories. Stellt generische CRUD-Operationen bereit und delegiert Query-Bau an <code>QueryBuilder</code> . Consuming Apps leiten pro Domänen-Entity eine Subklasse ab.
Command	JPA-Entity in Tabelle <code>egov_cmd_command</code> , die eine fachliche Schreiboperation repräsentiert. Enthält Benutzer, Zeitstempel, Aktion und Status-Lifecycle. Bildet die Grundlage des Audit-Trails.
CommandHandlerBase	Basis-Implementierung von <code>BaseExecutor</code> für Standard-Schreiboperationen. Consuming Apps leiten davon domänenspezifische Executors ab.
Consuming App	Fachapplikation einer Behörde, die das e-gov-framework als Maven-Dependency einbindet. Das Framework läuft in derselben JVM wie die Fachapplikation.
ContextInterceptor	Spring <code>HandlerInterceptor</code> , der bei jedem HTTP-Request die <code>userId</code> aus dem <code>EGOV_TOKEN</code> -Cookie extrahiert und im <code>DataHolder</code> ablegt. Bereinigt den <code>ThreadLocal</code> in <code>afterCompletion()</code> .
DataHolder	<code>ThreadLocal</code> -Kontext, der für die Dauer eines Requests die <code>userId</code> und Request-Parameter (<code>filter</code> , <code>sort</code> , <code>page</code> , <code>size</code> u.a.) hält. Wird von allen Framework-Komponenten gelesen.
DatabaseConfigurationService	Spring-Bean, der beim Start <code>egov.persistence.database.type</code> und <code>egov.persistence.database.naming.style</code> liest und Hibernate-Dialekt sowie Spalten-Namensschema konfiguriert.
DomainCommandFactory	Abstrakte Fabrik, die consuming Apps ableiten, um domänenspezifische Command-Instanzen zu erzeugen und mit Audit-Informationen zu befüllen.
EGOV_TOKEN	HTTP-Cookie, das die F5-Firewall nach erfolgreichem Azure-AD-Login setzt und die <code>userId</code> des authentifizierten Benutzers enthält. Das Framework führt keine eigene Token-Validierung durch.
ExecutionPlan	Enum auf dem Command, das steuert ob und wann Validierung und Ausführung stattfinden: <code>VALIDATE_BEFORE_EXECUTION</code> , <code>VALIDATE_BEFORE_EXECUTING_ANYWAY</code> , <code>VALIDATE_AFTER_EXECUTION</code> , <code>DONT_VALIDATE</code> , <code>DONT_EXECUTE</code> .
Filter-DSL	Eigenentwickelte Filtersprache für REST-Anfragen (z.B. <code>status==ACTIVE;name==M*</code>). <code>FilterParser</code> und Tokenizer übersetzen den String in einen Abstract Syntax Tree; <code>QueryBuilder</code> erzeugt daraus parameterisiertes SQL.
GenericExceptionHandler	Spring <code>@ControllerAdvice</code> , der alle <code>EGovException</code> -Subklassen abfängt und in ein strukturiertes <code>ResponseBody-JSON</code> umwandelt (inkl. <code>errorCode</code> , <code>exceptionId</code>).

Begriff	Definition
LUID	<i>Locally Unique Identifier</i> — zeitbasierte, sortierbare, cluster-sichere ID-Strategie des Moduls e-gov-luid. Kein Datenbankzugriff bei der Generierung; Eindeutigkeit über mehrere Pods via <code>segmentId</code> .
LUIDGenerator	Hibernate Identifizierungsgenerator, der LUIDs erzeugt. Thread-sicher (synchronized). Konfigurierbar über <code>egov.persistence.luid.*-Properties</code> .
ModifiableEntity	JPA-Basisklasse mit Audit-Feldern (<code>createdBy</code> , <code>modifiedBy</code> , Zeitstempel). Alle Framework-Entitäten und Domänen-Entitäten der consuming Apps erben davon.
NonTxExecutor	Variante von <code>BaseExecutor</code> ohne <code>@Transactional</code> — für Operationen, die ausserhalb einer Spring-Transaktion laufen müssen. Bekanntes Risiko (R-2); wird in Framework 2.0 adressiert.
PermissionBuilder	Komponente, die für eine <code>userId</code> , eine Entity-Klasse und eine <code>PermissionOperation</code> eine parametrisierte SQL-WHERE-Klausel erzeugt. Wird vom <code>QueryBuilder</code> automatisch in jede Leseabfrage eingebunden.
PermissionService	Service, der die aktiven Rollenberechtigungen eines Benutzers aus den IAM-Tabellen liest und <code>AllowedRecordsQuery</code> -Objekte zurückgibt.
QueryBuilder	Kernkomponente der Persistence-Schicht. Baut native SQL-Abfragen aus Filter-DSL, Sortierung, Paginierung und Permission-Filter zusammen.
RemoteCommandExecutor	<code>BaseExecutor</code> -Variante für verteilte Command-Ausführung über mehrere Kubernetes-Pods hinweg, koordiniert via ZooKeeper.
RuleSet	Datenbankgespeicherte Menge von Validierungsregeln für eine bestimmte Aktion. <code>RuleSetService</code> ermittelt den passenden <code>RuleSet</code> ; <code>ValidationEngine</code> wertet ihn gegen DTO-Felder aus.
RuleSetService	Service, der den für eine Kombination aus Aktion und Benutzerrolle passenden <code>RuleSet</code> aus den IAM-Tabellen ermittelt.
Row-based Security	Sicherheitskonzept, bei dem jede Datenbankabfrage automatisch um benutzer- und rollenspezifische WHERE-Klauseln erweitert wird. Kein Benutzer sieht Datensätze, für die keine <code>permission_definition</code> existiert. Opt-out statt Opt-in (ADR-004).
Soft-Delete	Archivierungsstrategie: Entitäten werden nicht physisch gelöscht, sondern als archiviert markiert. Umsetzung über <code>ArchivedModifiableEntity</code> .
UserIdProvider	Interface, das jede consuming App einmalig implementiert, um die <code>userId</code> aus dem <code>EGOV_TOKEN</code> -Cookie (oder einem anderen Auth-Mechanismus) zu extrahieren. Einziger Integrationspunkt für die Authentifizierung.
ValidationEngine	Komponente, die <code>RuleSet</code> -Validierungsregeln rekursiv gegen DTO-Felder auswertet und ein <code>validationTriple</code> (OK / FAILED + Fehlerbeschreibung) zurückgibt.
Vocabulary	MDM-Konzept für eine benannte Sammlung mehrsprachiger Code-Werte (z.B. Ortschaften, Anreden, Sprachen). Besteht aus <code>LanguageDefinition</code> -Einträgen und <code>MasterDataGenericEntity</code> -Zeilen pro Sprache.

12.2 Systemkomponenten und externe Systeme

Begriff	Definition
ArgoCD	GitOps-Deployment-Tool, das Kubernetes-Ressourcen aus einem Git-Repository synchronisiert. Consuming Apps werden über ArgoCD auf Kubernetes-Clustern der Behörden ausgerollt.
Artifactory	Maven-Artefakt-Registry von JFrog, betrieben von ti&m unter <code>bin.ti8m.ch/artifactory/</code> . Einziges Distributions-Repository für Framework-JARs; nicht Maven Central.
Azure AD	Microsoft Azure Active Directory — Identity Provider der Behörden. Stellt nach erfolgreicher Authentifizierung das <code>EGOV_TOKEN</code> -Cookie via F5 aus.

Begriff	Definition
Azure OpenAI	Cloud-Dienst von Microsoft für OpenAI-Modelle (gpt-4o). Wird vom Modul <code>e-gov-ai-rule-generation</code> für die LLM-gestützte Regelgenerierung genutzt.
F5	F5 BIG-IP Reverse Proxy / Firewall, der TLS terminiert und nach Azure-AD-Login den <code>EGOV_TOKEN</code> -Cookie setzt.
Flyway	Datenbank-Migrationstool. Das Framework liefert eigene Flyway-Skripte für IAM- und Command-Tabellen mit; consuming Apps ergänzen applikationsspezifische Skripte.
Spring AI	Spring-Framework-Erweiterung als Abstraktionsschicht für LLM-Integration. Ermöglicht späteren Wechsel des LLM-Providers ohne Code-Änderung. Aktuell als Milestone-Release eingesetzt (→ Risiko R-4).
ZooKeeper	Verteiltes Koordinationssystem (Apache). Optional; wird für <code>RemoteCommandExecutor</code> genutzt, wenn Commands über mehrere Pod-Instanzen hinweg verteilt ausgeführt werden sollen.

12.3 Abkürzungen

Abkürzung	Bedeutung	Kontext
ADR	Architecture Decision Record	Sektion 9: dokumentiert wichtige Architekturentscheidungen im Nygard-Format
arc42	Architecture 42	Template-Standard für Softwarearchitektur-Dokumentation (https://arc42.org)
BOM	Bill of Materials	Maven-Konzept; e-gov-bom steuert die Versionen aller Framework-Abhängigkeiten zentral
CRUD	Create, Read, Update, Delete	Grundlegende Datenbankoperationen; <code>BaseRepositoryImpl</code> stellt sie generisch bereit
DDL	Data Definition Language	SQL-Befehle zur Schemadefinition; Framework liefert Flyway-DDL für IAM- und Command-Tabellen
DSL	Domain-Specific Language	Im Framework: Filter-DSL für REST-Anfragen (z.B. <code>status==ACTIVE;name==M*</code>)
DTO	Data Transfer Object	Datenobjekt für REST-Schnittstellen; trägt <code>@RuleSet</code> - und <code>@Validation</code> -Annotationen
IAM	Identity and Access Management	Framework-Modul <code>ch.ti8m.egov.framework.iam</code> : Rollen, <code>RuleSets</code> , <code>Permissions</code>
JDBC	Java Database Connectivity	Low-level Java API für Datenbankzugriff; wird im <code>QueryBuilder</code> für native SQL-Abfragen genutzt
JPA	Java Persistence API	ORM-Standard; wird für Entity-Mapping und Schreiboperationen eingesetzt
LTS	Long-Term Support	Java-Release-Kategorie; Java 21 ist die aktuelle LTS-Version (Randbedingung)
LUID	Locally Unique Identifier	Zeitbasierte, sortierbare, cluster-sichere ID-Strategie des e-gov-luid-Moduls

Abkürzung	Bedeutung	Kontext
MDM	Master Data Management	Modul e-gov-mdm: verwaltet mehrsprachige Stammdaten (Vocabularies, Codelisten)
REST	Representational State Transfer	HTTP-basiertes API-Stil; alle Framework-Endpunkte sind REST-konform und via SpringDoc/OpenAPI dokumentiert
RBAC	Role-Based Access Control	Berechtigungskonzept: Zugriff wird anhand von Rollen gesteuert; Basis des IAM-Moduls
SemVer	Semantic Versioning	Versionierungsschema MAJOR.MINOR.PATCH; Breaking Changes → Major-Bump; automatisiert via Conventional Commits + Semantic Release
ULID	Universally Unique Lexicographically Sortable Identifier	Alternative ID-Strategie in e-gov-luid; ms-präzise, ohne segmentId